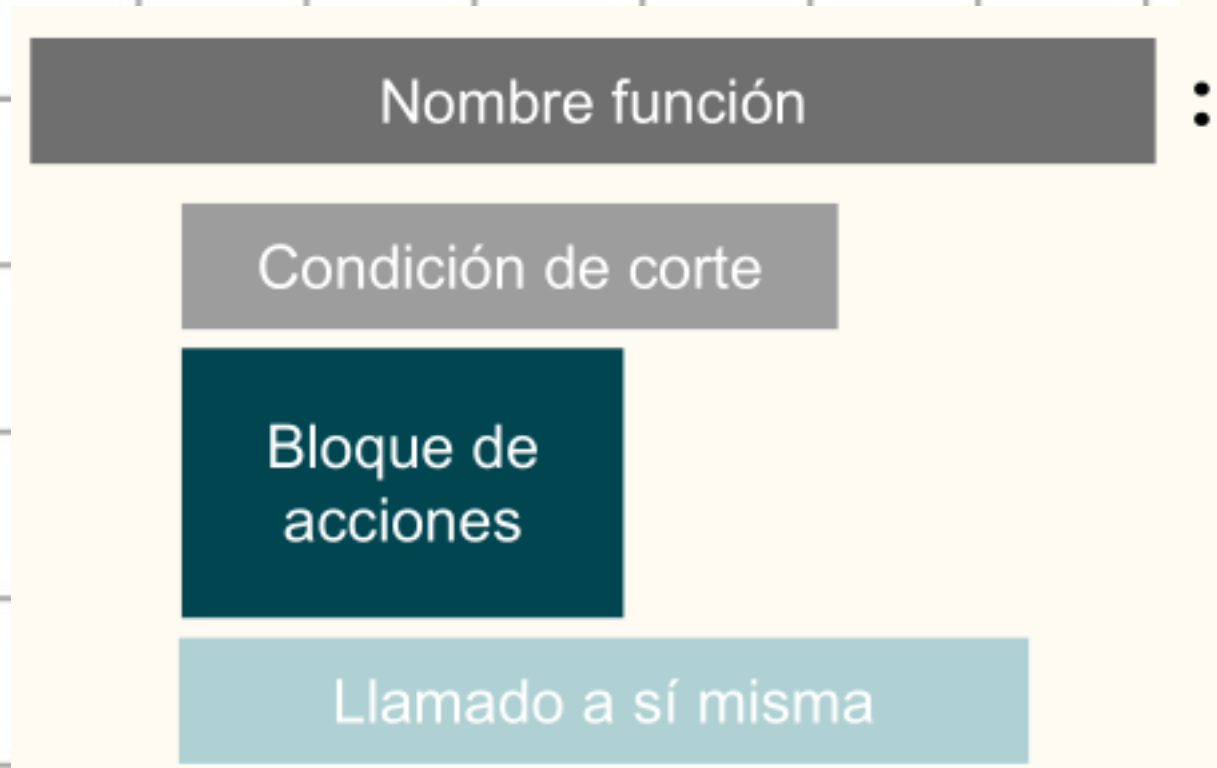


Recursividad

Las funciones recursivas son funciones en las que, dentro de la definición, tienen por lo menos un llamado a sí mismas.

```
def vivir_mi_vida():
    despertarme()
    desayunar()
    trabajar()
    almorzar()
    estudiar()
    dormirme()
    vivir_mi_vida()
```

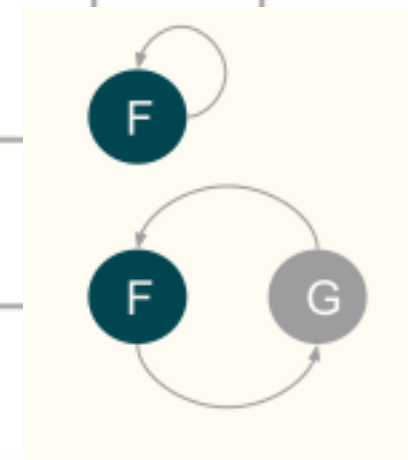
Partes de la recursividad



Clasificación

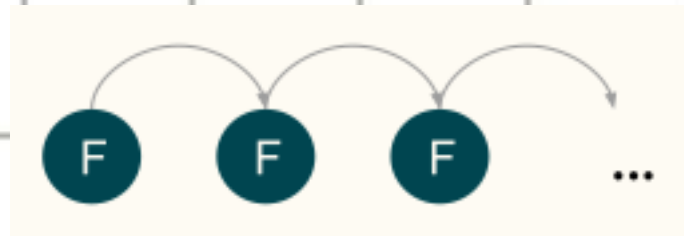
- Por la forma de invocación

- Directa
- Indirecta
- Anidada

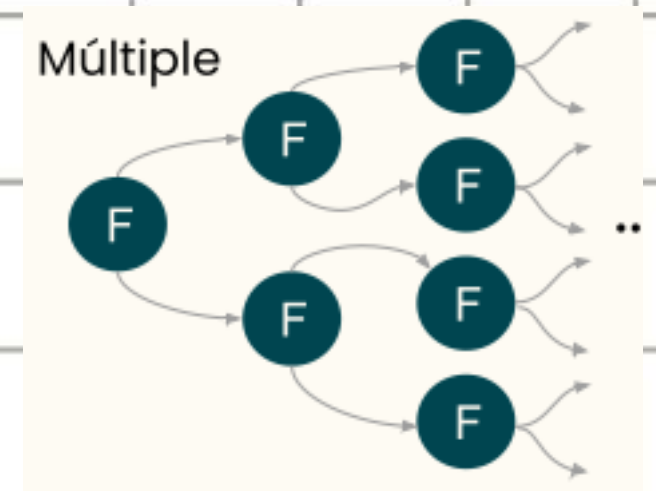

$$F(N + F(N + 3))$$

- Por la cantidad de llamados

- simple



- Multiple



Ejemplos

POTENCIA

$$\text{base}^{\text{exponente}} = \underbrace{\text{base} * \text{base} * \dots * \text{base}}_{\text{exponente veces}}$$

```
public int potencia(base, exponente){
    if (exponente == 0){
        return 1;
    }
    return base * potencia(base, exponente - 1);
}
```

FIBONACCI

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

$$\text{Fibo}(\text{orden}) = \text{Fibo}(\text{orden} - 1) + \text{Fibo}(\text{orden} - 2)$$

```
public int fibonacci(orden){
    if (orden == 0){
        return 0;
    }
    if (orden == 1){
        return 1;
    }
    return fibonacci(orden - 1) + fibonacci(orden - 2);
}
```

Ejercicio Realizar un método que resuelva el factorial de un número con recursividad. Clasifique la solución.

Solución

```
public int factorial(numero) {  
    if ( numero == 0 || numero == 1) {  
        return 1;  
    }  
    return numero * factorial(numero - 1);  
}
```

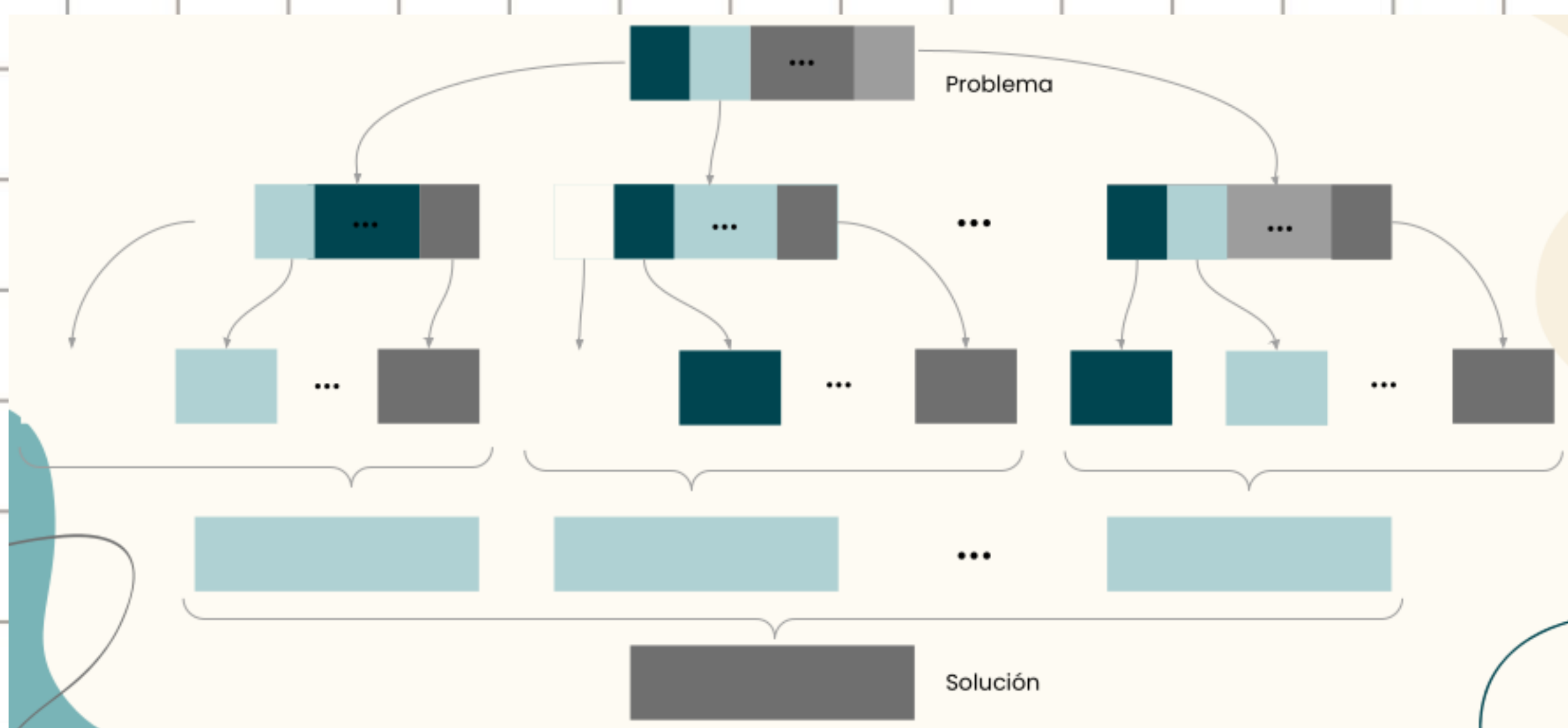
Recursión directa simple.

Teoría DyV

Definición Si el tamaño del problema a tratar es $N > L$, siendo L un valor arbitrario, se debe dividir el problema en m subproblemas no solapados y que tengan la misma estructura que el problema original. Luego, a cada uno de esos m subproblemas se los dividirá nuevamente hasta que cada uno alcance un tamaño L o menor, los cuales se resolverán por cualquier otro método.

Por último, la solución del problema original se obtiene por combinación de las m soluciones obtenidas de los subproblemas en que se lo había dividido.

DyC : algoritmo gráfico



EjemplosPotenciaCalcular 2^{64}

```
int potencia(int base, int exponente) {
    int resultado = 1;

    for (int i = 0; i < exponente; i++)
        resultado *= base;

    return resultado;
}
```

```
int potencia(int base, int exponente) {
    if (exponente == 0)
        return 1;

    return base * potencia(base, exponente - 1);
}
```

64 iteraciones

$i = 0 \rightarrow \text{resultado} = 2$
 $i = 1 \rightarrow \text{resultado} = 4$
 $i = 2 \rightarrow \text{resultado} = 8$
 $\dots$
 $i = 63 \rightarrow \text{resultado} = 1,844674407 \times 10^{19}$

Llamado 1 $\rightarrow$ exponente = 63
 Llamado 2 $\rightarrow$ exponente = 62
 $\dots$
 Llamado 64 $\rightarrow$ exponente = 0 (C.B.)
 El resultado queda $1,844674407 \times 10^{19}$

64 llamados

Aplicando DyC buscamos reducir la cantidad de iteraciones / llamados.

```
int potencia(int base, int exponente) {
    if (exponente == 0)
        return 1;

    else if (exponente == 1)
        return base;

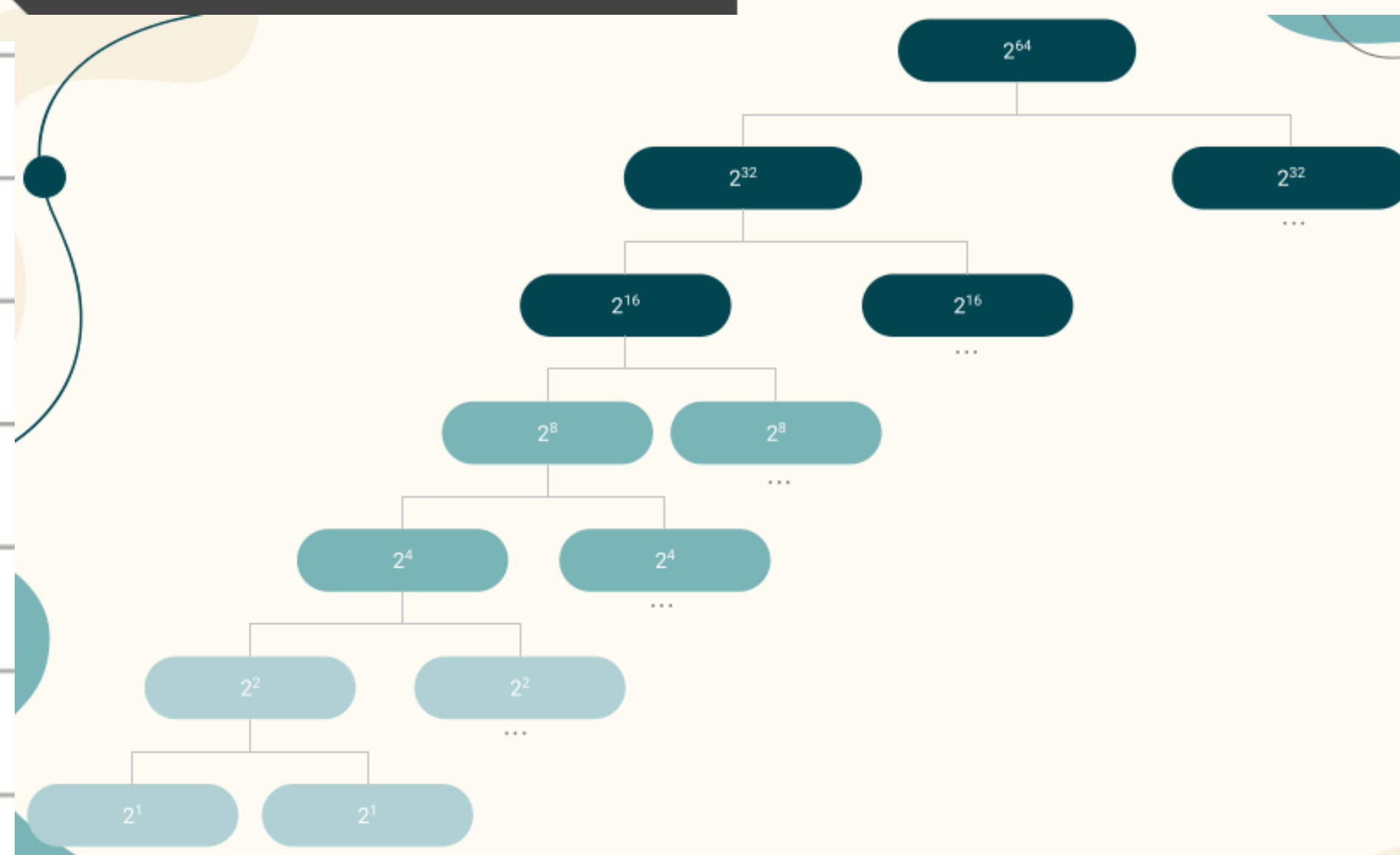
    int resultado = potencia(base, exponente / 2);
    resultado *= resultado;

    if ((exponente % 2) == 1)
        resultado *= base;

    return resultado;
}
```

Llamado 1 $\rightarrow$ exponente = 32
 Llamado 2 $\rightarrow$ exponente = 16
 Llamado 3 $\rightarrow$ exponente = 8
 Llamado 4 $\rightarrow$ exponente = 4
 Llamado 5 $\rightarrow$ exponente = 2
 Llamado 6 $\rightarrow$ exponente = 1 (C.B.)

6 llamados



los algoritmos de DyC si bien suelen asociarse con recursividad, Pueden resolverse de forma iterativa.

```
int potencia(int base, int exponente) {
    int resultado = base;

    for (int i = exponente - 1; i > 0; i /= 2) {
        if ((exponente % 2) == 1)
            resultado *= base;

        resultado *= resultado;
    }
    return resultado;
}
```

SIN RECURSIVIDAD

Búsqueda Binaria

